

Construindo um Projeto Ágil no GitHub: Da Gestão ao Controle de Qualidade

Documento Teórico — Projeto TechFlow TaskManager

Disciplina: Engenharia de Software

Aluna: Karoline Cristina Amaral Ribeiro

RA: 83533

Repositório no GitHub: <https://github.com/karolinecristinay-eng/techflow-taskmanager>

1. Descrição do Projeto e Escopo Inicial

A TechFlow Solutions, empresa fictícia especializada em soluções de software, foi contratada por uma startup de logística para desenvolver um sistema de gerenciamento de tarefas baseado em metodologias ágeis. O objetivo do sistema é permitir o acompanhamento do fluxo de trabalho em tempo real, a priorização de tarefas críticas e o monitoramento do desempenho da equipe.

O escopo inicial do projeto (TechFlow TaskManager) definiu um sistema web simples de CRUD (Create, Read, Update, Delete) de tarefas, contendo:

- Cadastro, listagem, atualização e remoção de tarefas;
- Organização das tarefas em três status equivalentes às colunas do quadro Kanban: A Fazer, Em Progresso e Concluído;
- Uma interface web simples para visualização do quadro e uma API REST para integração com outros sistemas.

O projeto foi versionado em um repositório público no GitHub, disponível em <https://github.com/karolinecristinay-eng/techflow-taskmanager>, com 11 commits registrados e 22 testes automatizados passando (Pytest).

2. Metodologia Ágil Utilizada

Para este projeto, adotou-se o Kanban como metodologia ágil principal, por ser adequado a um fluxo de trabalho contínuo (sem iterações fixas de tempo, como ocorre no Scrum), o que se encaixa bem no perfil de uma equipe de manutenção e evolução constante de um sistema de tarefas.

O quadro Kanban foi expandido para 5 colunas — Backlog, A Fazer, Em Progresso, Em Revisão/Testes e Concluído — em vez das 3 colunas mínimas exigidas, para tornar explícita a etapa de controle de qualidade entre a implementação e a entrega de cada tarefa (ver Seção 4.3). O fluxo de trabalho segue os princípios:

- Visualização do fluxo: todo o trabalho fica visível no quadro, facilitando a comunicação da equipe;
- Limitação do trabalho em progresso: evita-se iniciar muitas tarefas simultaneamente sem concluí-las;
- Entregas incrementais: cada tarefa concluída gera um ou mais commits, integrados por meio do pipeline de CI;
- Melhoria contínua: o processo é ajustado ao longo do projeto, como demonstrado na mudança de escopo descrita na Seção 6.

3. A Importância da Modelagem na Engenharia de Software

A modelagem é a etapa da Engenharia de Software em que abstraímos o problema do mundo real (o sistema de gerenciamento de tarefas de uma equipe ágil) em representações visuais e estruturadas, antes de escrever qualquer código. Ela é importante por diversos motivos:

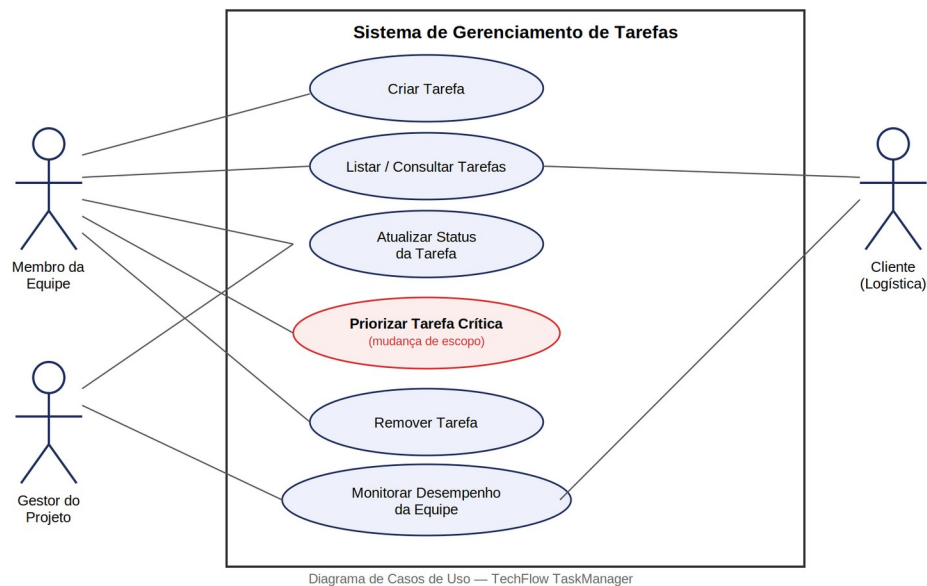
- Comunicação: diagramas como o de Casos de Uso e o de Classes permitem que desenvolvedores, gestores e clientes compartilhem uma mesma visão do sistema, reduzindo ambiguidades;

- Antecipação de problemas: erros de design identificados na modelagem custam muito menos para corrigir do que erros encontrados após a implementação;
- Rastreabilidade: a modelagem serve de referência para validar se a implementação atende aos requisitos levantados junto ao cliente;
- Facilidade de manutenção: um sistema bem modelado, com responsabilidades bem definidas entre suas classes, é mais fácil de estender — como ficou evidente na Seção 6, quando um novo atributo (prioridade) foi adicionado ao modelo sem impactar as demais funcionalidades.

4. Diagramas e Quadro Kanban

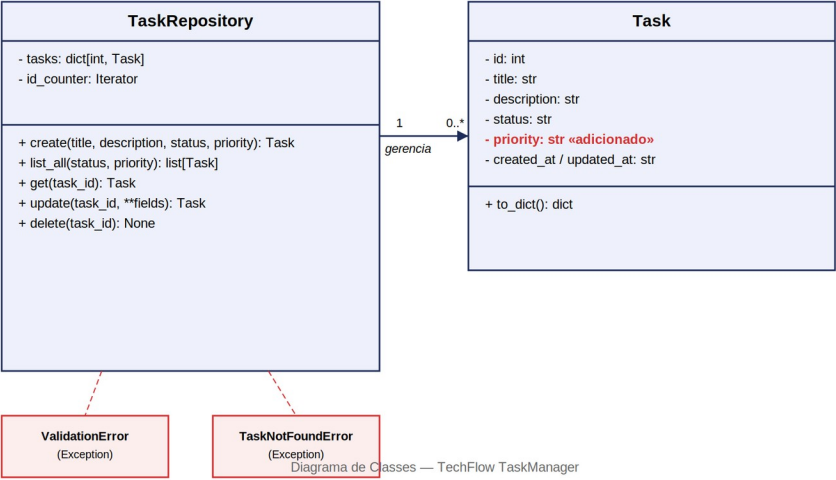
4.1 Diagrama de Casos de Uso

O diagrama abaixo representa os principais atores do sistema (Membro da Equipe, Gestor do Projeto e Cliente da startup de logística) e as funcionalidades (casos de uso) que cada um pode acionar. O caso de uso "Priorizar Tarefa Crítica" foi incorporado ao sistema por meio da mudança de escopo detalhada na Seção 6.



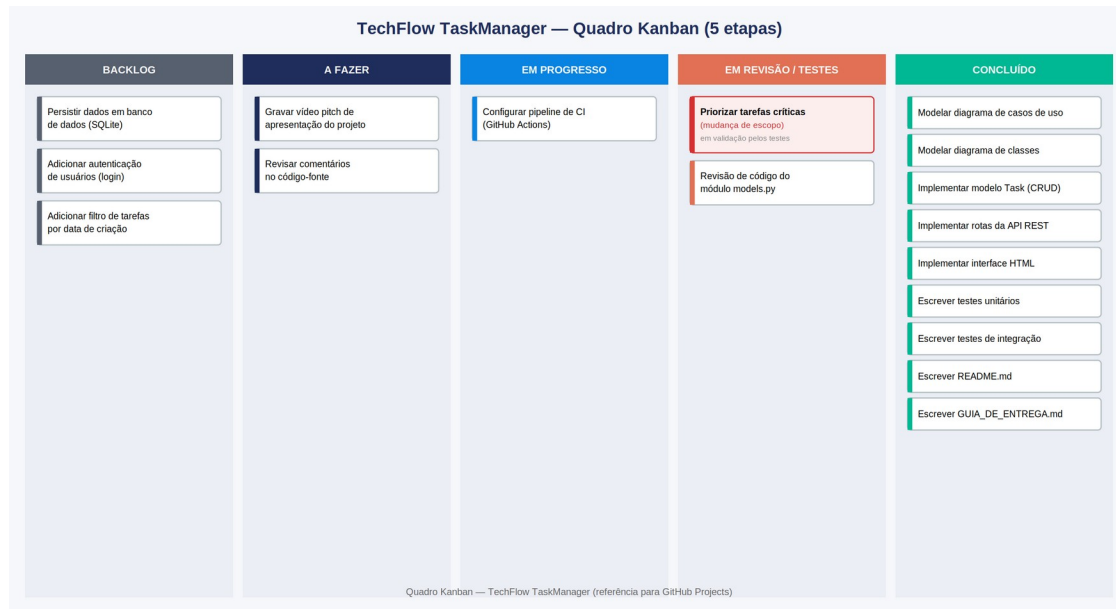
4.2 Diagrama de Classes

O diagrama de classes evidencia a separação entre a camada de domínio (Task e TaskRepository) e o tratamento de erros (ValidationError e TaskNotFoundError), independente da camada web (Flask). O atributo priority é o elemento incorporado na mudança de escopo.



4.3 Quadro Kanban (versão expandida, 5 etapas)

Diferentemente do quadro mínimo de 3 colunas exigido no enunciado (A Fazer, Em Progresso, Concluído), optou-se por uma versão expandida com 5 colunas: Backlog (ideias futuras ainda não priorizadas), A Fazer, Em Progresso, Em Revisão/Testes (nenhuma tarefa é considerada concluída sem antes passar pelos testes automatizados) e Concluído. Essa divisão reforça, na prática, a ligação entre gestão ágil e controle de qualidade — os dois eixos centrais deste trabalho.



O card "Priorizar tarefas críticas (mudança de escopo)" está posicionado em "Em Revisão/Testes", pois sua implementação já foi feita no código (atributo priority em models.py), mas depende da validação contínua da suíte de testes automatizados a cada novo commit.

5. Testes Automatizados Utilizados

O controle de qualidade do projeto foi implementado com a biblioteca Pytest, cobrindo duas camadas do sistema:

- Testes unitários (tests/test_models.py): validam isoladamente a regra de negócio do repositório de tarefas — criação, listagem com filtros e ordenação por prioridade, atualização, remoção e o tratamento de erros de validação (ValidationError) e de tarefas inexistentes (TaskNotFoundError);
- Testes de integração (tests/test_app.py): utilizam o cliente de testes do próprio Flask para simular requisições HTTP reais à API REST (/api/tasks), validando os códigos de status HTTP retornados (200, 201, 400, 404, 204) em cada cenário.

No total, o projeto conta com 22 testes automatizados, todos passando. Esses testes são executados automaticamente pelo pipeline de Integração Contínua configurado em .github/workflows/ci.yml, disparado a cada push e pull request na branch main.

6. Justificativa sobre a Mudança de Escopo

Durante o desenvolvimento, identificou-se que o escopo inicial — um CRUD simples com apenas título, descrição e status — não atendia integralmente ao requisito do cliente de "priorizar tarefas críticas", citado explicitamente no desafio original. Uma equipe de logística lida com urgências distintas a cada tarefa (por exemplo, um atraso de entrega tem impacto muito maior do que um ajuste de rotina), e sem um campo de prioridade essa diferenciação não era possível.

Mudança implementada: adição do atributo priority (com os valores low, medium, high e critical) ao modelo Task, incluindo validação de valores permitidos e ordenação automática da listagem de tarefas, de forma que tarefas críticas apareçam sempre no topo do quadro.

Condução da mudança dentro do processo ágil:

- 1) Criação de um novo card no quadro Kanban, na coluna "A Fazer", descrevendo a necessidade identificada;
- 2) Movimentação do card para "Em Progresso" durante a implementação da funcionalidade;
- 3) Commit implementando a mudança no atributo priority do modelo Task;
- 4) Atualização do README.md do repositório com a justificativa da mudança (reproduzida nesta seção);
- 5) Card movido para "Em Revisão/Testes", onde permanece até a suíte de testes confirmar a nova regra de negócio.

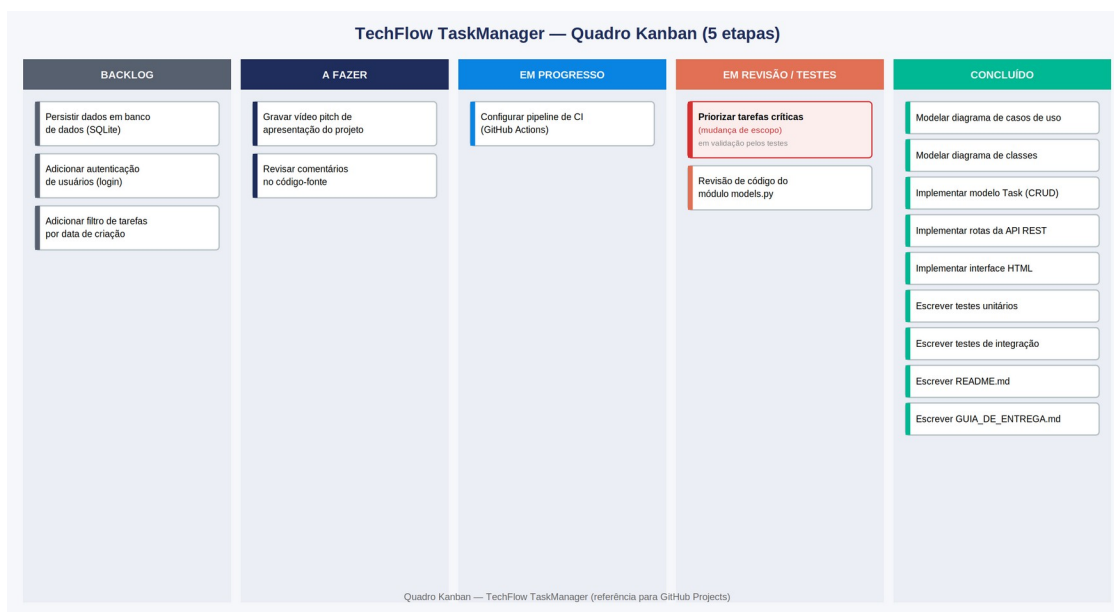
Esse processo demonstra, na prática, como uma equipe conduzida com Kanban absorve mudanças de escopo de forma incremental e rastreável, sem comprometer o que já havia sido entregue.

7. Prints Comentados do GitHub

As imagens a seguir foram elaboradas a partir dos dados reais do repositório público <https://github.com/karolinecristinay-eng/techflow-taskmanager> (hashes de commit, mensagens e datas reais, extraídos diretamente do histórico do Git), reconstruídas no layout do GitHub para uso neste documento.

7.1 Quadro Kanban

Representação do quadro Kanban de 5 colunas descrito na Seção 4.3, a ser reproduzido na aba Projects do repositório.



7.2 Histórico de Commits

Histórico real dos 11 commits do repositório, do mais recente ("commit final", 171bef6) ao commit inicial (458d839), demonstrando o versionamento incremental do projeto ao longo do desenvolvimento.

karolinecristinay-eng / techflow-taskmanager		Branch: main · 11 commits
commit final karolinecristinay-eng	171bef6 · 11 jul 2026	
commit 9 karolinecristinay-eng	9c7faf6 · 11 jul 2026	
commit 8 karolinecristinay-eng	707e049 · 11 jul 2026	
commit 7 karolinecristinay-eng	25dff8c4 · 11 jul 2026	
commit 6 karolinecristinay-eng	f239ce0 · 11 jul 2026	
commit 5 karolinecristinay-eng	83a7025 · 11 jul 2026	
commit 4 karolinecristinay-eng	7f06ab5 · 11 jul 2026	
commit 3 karolinecristinay-eng	0d59674 · 11 jul 2026	
commit 2 karolinecristinay-eng	8d93903 · 11 jul 2026	
commit inicial karolinecristinay-eng	70d78f5 · 11 jul 2026	
Initial commit karolinecristinay-eng	458d839 · 10 jul 2026	

7.3 Workflow de CI (GitHub Actions)

Resultado esperado da execução do pipeline de Integração Contínua após o commit e push do arquivo `.github/workflows/ci.yml` (incluído no pacote desta entrega), rodando os 22 testes automatizados do projeto.

karolinecristinay-eng / techflow-taskmanager

Actions

CI - Testes Automatizados

✓

ci: configura pipeline de integração contínua com GitHub Actions

main · pytest -v --maxfail=1 · concluído com sucesso

≈ 18s

Jobs / test

✓ Checkout do código

✓ Configurar Python 3.11

✓ Instalar dependências (pip install -r requirements.txt)

✓ Rodar testes automatizados (Pytest) — 22 passed

✓ Verificar qualidade básica do código (py_compile)

Execução esperada após o push do arquivo `.github/workflows/ci.yml` incluído nesta entrega

8. Considerações Finais

Este projeto permitiu aplicar, de forma integrada e prática, os principais pilares da Engenharia de Software: modelagem (diagramas UML), versionamento (Git/GitHub, com 11 commits e histórico rastreável), gestão ágil (Kanban expandido de 5 colunas), controle de qualidade (22 testes automatizados e pipeline de CI) e gestão de mudanças (adaptação de escopo documentada e rastreável). A combinação dessas práticas é o que diferencia um projeto de software amador de um processo profissional, capaz de evoluir com segurança e transparência ao longo do tempo.